

ESP32S3 CAM开发板代码:

```
// send_img_huawei.ino - MaaS方案完整版 (ESP32-S3 CAM版本) - 修正版

#include <WiFi.h>
#include <Wire.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include "esp_camera.h"
#include <base64.h>

// WiFi配置
#define WIFI_SSID ""
#define WIFI_PASSWD ""

// 华为云物联网设备信息
#define DEVICE_ID "690343cb0a9ab42ae589dd21_CAM01"
#define DEVICE_SECRET "lego0369"
#define CLIENT_ID "690343cb0a9ab42ae589dd21_CAM01_0_0_2025103012"
#define MQTT_USER "690343cb0a9ab42ae589dd21_CAM01"
#define MQTT_PASSWORD
"59f8a64dc6142dd640d9dc375828825734f5943accfc0864686a77c39b4aec7e"

// 华为云MQTT配置
#define MQTT_SERVER "943f583bff.st1.iotda-device.cn-east-3.myhuaweicloud.com"
#define MQTT_PORT 1883

// MQTT主题定义
#define SERVICE_ID "\"ESP32CAM01\""
#define MQTT_BODY_FORMAT "{\"services\": [{\"service_id\": \"SERVICE_ID\", \"properties\": {\"%s\"}}]}"
#define MQTT_TOPIC_REPORT "$oc/devices/" DEVICE_ID "/sys/properties/report"
#define MQTT_TOPIC_COMMANDS "$oc/devices/" DEVICE_ID "/sys/commands/#"
#define MQTT_TOPIC_CMD_RESPONSE "$oc/devices/" DEVICE_ID "/sys/commands/response/request_id="
#define MQTT_TOPIC_MESSAGES "$oc/devices/" DEVICE_ID "/sys/messages/#"

// Base64分片配置
#define BASE64_CHUNK_SIZE 800

// 果云ESP32-S3 CAM 引脚配置
#define PWDN_GPIO_NUM -1
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM 15
#define SIOD_GPIO_NUM 4
#define SIOC_GPIO_NUM 5
#define Y9_GPIO_NUM 16
#define Y8_GPIO_NUM 17
#define Y7_GPIO_NUM 18
#define Y6_GPIO_NUM 12
#define Y5_GPIO_NUM 10
#define Y4_GPIO_NUM 8
#define Y3_GPIO_NUM 9
#define Y2_GPIO_NUM 11
#define VSYNC_GPIO_NUM 6
#define HREF_GPIO_NUM 7
#define PCLK_GPIO_NUM 13
#define LED_GPIO_NUM 21
```

```

static camera_config_t camera_config = {
    .pin_pwdn = PWDN_GPIO_NUM,
    .pin_reset = RESET_GPIO_NUM,
    .pin_xclk = XCLK_GPIO_NUM,
    .pin_sccb_sda = SIOD_GPIO_NUM,
    .pin_sccb_scl = SIOC_GPIO_NUM,
    .pin_d7 = Y9_GPIO_NUM,
    .pin_d6 = Y8_GPIO_NUM,
    .pin_d5 = Y7_GPIO_NUM,
    .pin_d4 = Y6_GPIO_NUM,
    .pin_d3 = Y5_GPIO_NUM,
    .pin_d2 = Y4_GPIO_NUM,
    .pin_d1 = Y3_GPIO_NUM,
    .pin_d0 = Y2_GPIO_NUM,
    .pin_vsync = VSYNC_GPIO_NUM,
    .pin_href = HREF_GPIO_NUM,
    .pin_pclk = PCLK_GPIO_NUM,
    .xclk_freq_hz = 20000000,
    .ledc_timer = LEDC_TIMER_0,
    .ledc_channel = LEDC_CHANNEL_0,
    .pixel_format = PIXFORMAT_JPEG,
    .frame_size = FRAMESIZE_SVGA,
    .jpeg_quality = 8,
    .fb_count = 2
};

WiFiClient espClient;
PubSubClient mqttClient(espClient);

// ===== 系统状态变量 =====
unsigned long lastCaptureTime = 0;
bool isCapturing = false;
String currentRequestId = "";

// 定时上报相关
unsigned long lastReportTime = 0;
const unsigned long REPORT_INTERVAL = 30000; // 30秒上报一次

// 系统统计变量
int photoCount = 0;
int messageCount = 0;
int errorCount = 0;
int aiDetectionCount = 0;
int commandCount = 0;
unsigned long systemUptime = 0;
String lastActivity = "系统启动";

// 控制参数变量
bool deviceSwitch = true;
int captureInterval = 30;
bool autoUpload = true;
bool enableAI = true;

// ===== 图片中显示的变量（需要定时上报） =====
String humidity = "54"; // 湿度
String feelsLike = "16"; // 体感温度
String scheduleTitle = "军事理论"; // 最近日程名称
String windScale = "3"; // 风力
String visibility = "7"; // 能见度
String countdownString = "8m"; // 日程倒计时

```

```

String temp = "18";                // 温度
String windDir = "东风";           // 风向
String aiResult = "暂无AI识别结果"; // AI识别结果
String condition = "阴";           // 天气状况
String windSpeed = "15";           // 风速
String scheduleTime = "11-10 13:25"; // 最近日程时间

// ===== AI结果变量 =====
String aiResultType = "none";
unsigned long aiTimestamp = 0;
bool hasAIResult = false;
unsigned long lastAIReportTime = 0;

// ===== 其他变量 =====
String city = "杭州";
String updateTime = "N/A";
bool hasWeatherData = false;
bool hasSchedule = false;
String scheduleExactTime = "";
String scheduleLocation = "";
String scheduleDescription = "";
String scheduleUpdateTime = "";

void setup() {
    Serial.begin(115200);
    Serial.println("🚀 ESP32-S3 CAM MaaS AI识别系统启动");

    // 检查PSRAM
    if(psramFound()){
        Serial.println("✅ PSRAM检测成功");
        camera_config.frame_size = FRAMESIZE_SVGA;
        camera_config.jpeg_quality = 10;
        camera_config.fb_count = 2;
    } else {
        Serial.println("⚠️ 未检测到PSRAM, 使用基础配置");
    }

    init_wifi(WIFI_SSID, WIFI_PASSWD);
    camera_init();

    // 初始化LED引脚
    pinMode(LED_GPIO_NUM, OUTPUT);
    digitalWrite(LED_GPIO_NUM, LOW);

    mqttClient.setServer(MQTT_SERVER, MQTT_PORT);
    mqttClient.setCallback(mqtt_callback);

    Serial.println("✅ 系统初始化完成");
}

void loop() {
    if (!mqttClient.connected()) {
        Serial.println("⚠️ MQTT连接断开, 尝试重连");
        mqtt_check_connect();
    }
    mqttClient.loop();

    // 定时上报系统状态 (不包含AIResult)
    if (!isCapturing && millis() - lastReportTime >= REPORT_INTERVAL) {
        Serial.println("📺 定时上报系统状态");
        bool success = reportSystemStatus();
    }
}

```

```

        if (success) {
            lastReportTime = millis();
        } else {
            Serial.println("❌ 状态上报失败，下次再试");
        }
    }

    delay(100);
}

// ===== 初始化WiFi =====
void init_wifi(const char *ssid, const char *password) {
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    Serial.print("🔄 连接WiFi");

    int attempts = 0;
    while (WiFi.status() != WL_CONNECTED && attempts < 20) {
        delay(500);
        Serial.print(".");
        attempts++;
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("\n✅ WiFi连接成功");
        Serial.print("📄 IP地址: ");
        Serial.println(WiFi.localIP());
    } else {
        Serial.println("\n❌ WiFi连接失败");
        errorCount++;
    }
}

// ===== 初始化摄像头 =====
esp_err_t camera_init() {
    esp_err_t err = esp_camera_init(&camera_config);
    if (err != ESP_OK) {
        Serial.printf("❌ 摄像头初始化失败，错误代码: 0x%x\n", err);
        errorCount++;
        return err;
    }

    sensor_t *s = esp_camera_sensor_get();
    if (s != NULL) {
        Serial.println("✅ 摄像头初始化成功");
    }

    return ESP_OK;
}

// ===== MQTT回调函数 =====
void mqtt_callback(char *topic, byte *payload, unsigned int length) {
    Serial.print("📬 收到消息 [");
    Serial.print(topic);
    Serial.print("] ");

    payload[length] = '\0';
    String message = String((char*)payload);
    Serial.println(message);

    if (strstr(topic, "sys/messages")) {

```

```

        handleAIResultMessage(message);
    }
    else if (strstr(topic, "sys/commands")) {
        handleCommandMessage(message, topic);
    }

    messageCount++;
}

// ===== 处理AI结果消息 =====
void handleAIResultMessage(String message) {
    Serial.println("\n🤖 === AI识别结果 ===");

    DynamicJsonDocument doc(2048);
    DeserializationError error = deserializeJson(doc, message);

    String result = "未知";

    if (!error) {
        if (doc.containsKey("content")) {
            String content = doc["content"];
            DynamicJsonDocument contentDoc(2048);
            DeserializationError contentError = deserializeJson(contentDoc, content);

            if (!contentError && contentDoc.containsKey("AIResult")) {
                result = contentDoc["AIResult"].as<String>();
            } else {
                result = content;
            }
        }
        else if (doc.containsKey("AIResult")) {
            result = doc["AIResult"].as<String>();
        }
        else {
            result = message;
        }
    } else {
        result = message;
    }

    // 存储AI结果
    aiResult = result;
    aiResultType = "message";
    aiTimestamp = millis();
    hasAIResult = true;

    aiDetectionCount++;
    lastActivity = "AI识别: " + result.substring(0, 20) + "...";

    // ✅ 收到AI结果后立即单独上报AIResult
    Serial.println("🚀 收到AI结果, 立即单独上报AIResult...");
    reportAIResultOnly();

    Serial.println("=====\n");
}

// ===== 处理命令消息 =====
void handleCommandMessage(String message, char* topic) {
    DynamicJsonDocument doc(1024);
    DeserializationError error = deserializeJson(doc, message);

```

```

if (error) {
    Serial.println("❌ JSON解析失败");
    errorCount++;
    return;
}

String commandName = doc["command_name"].as<String>();
Serial.printf("🔔 收到命令: %s\n", commandName.c_str());

if (commandName == "AIResult") {
    handleAIResultCommand(doc["paras"]);
    sendCommandResponse(topic);
}
else if (commandName == "WeatherUpdate") {
    handleWeatherCommand(doc["paras"]);
    sendCommandResponse(topic);
}
else if (commandName == "SystemControl") {
    handleSystemControlCommand(doc["paras"]);
    sendCommandResponse(topic);
}
else if (commandName == "TakePhoto") {
    handleTakePhotoCommand(doc["paras"], topic);
}
else if (commandName == "ScheduleUpdate") {
    handleScheduleCommand(doc["paras"]);
    sendCommandResponse(topic);
}
else {
    Serial.printf("⚠️ 未知命令类型: %s\n", commandName.c_str());
    sendCommandResponse(topic);
}

commandCount++;
lastActivity = "执行命令: " + commandName;
}

// ===== 天气命令处理 =====
void handleWeatherCommand(JsonObject paras) {
    Serial.println("🌤️ 处理天气更新命令");

    if (paras.containsKey("Humidity")) {
        humidity = paras["Humidity"].as<String>();
        Serial.printf("☑️ 设置湿度: %s%\n", humidity.c_str());
    }
    if (paras.containsKey("FeelsLike")) {
        feelsLike = paras["FeelsLike"].as<String>();
        Serial.printf("☑️ 设置体感温度: %s°C\n", feelsLike.c_str());
    }
    if (paras.containsKey("Temp")) {
        temp = paras["Temp"].as<String>();
        Serial.printf("☑️ 设置温度: %s°C\n", temp.c_str());
    }
    if (paras.containsKey("Condition")) {
        condition = paras["Condition"].as<String>();
        Serial.printf("☑️ 设置天气: %s\n", condition.c_str());
    }
    if (paras.containsKey("WindScale")) {
        windScale = paras["WindScale"].as<String>();
        Serial.printf("☑️ 设置风力: %s级\n", windScale.c_str());
    }
}

```

```

if (paras.containsKey("WindDir")) {
    windDir = paras["WindDir"].as<String>();
    Serial.printf("☑ 设置风向: %s\n", windDir.c_str());
}
if (paras.containsKey("WindSpeed")) {
    windSpeed = paras["WindSpeed"].as<String>();
    Serial.printf("☑ 设置风速: %s\n", windSpeed.c_str());
}
if (paras.containsKey("Visibility")) {
    visibility = paras["Visibility"].as<String>();
    Serial.printf("☑ 设置能见度: %s\n", visibility.c_str());
}

hasWeatherData = true;
Serial.println("☑ 天气数据更新完成");
}

// ===== 系统控制命令处理 =====
void handleSystemControlCommand(JsonObject paras) {
    Serial.println("⚙ 处理系统控制命令");

    if (paras.containsKey("DeviceSwitch")) {
        deviceSwitch = paras["DeviceSwitch"];
        Serial.printf("🔌 设备开关: %s\n", deviceSwitch ? "开启" : "关闭");
    }
    if (paras.containsKey("CaptureInterval")) {
        captureInterval = paras["CaptureInterval"];
        Serial.printf("📷 拍照间隔: %d秒\n", captureInterval);
    }
    if (paras.containsKey("AutoUpload")) {
        autoUpload = paras["AutoUpload"];
        Serial.printf("📶 自动上传: %s\n", autoUpload ? "开启" : "关闭");
    }
    if (paras.containsKey("EnableAI")) {
        enableAI = paras["EnableAI"];
        Serial.printf("🧠 AI识别: %s\n", enableAI ? "开启" : "关闭");
    }
}

// ===== 拍照命令处理 =====
void handleTakePhotoCommand(JsonObject paras, char* topic) {
    Serial.println("📷 处理拍照命令");

    bool takePhoto = true;
    if (paras.containsKey("TakePhoto")) {
        takePhoto = paras["TakePhoto"];
    }

    sendCommandResponseImmediately(topic);

    if (takePhoto && !isCapturing && deviceSwitch) {
        Serial.println("🌀 开始执行拍照任务...");
        captureAndSendForAI();
    } else {
        if (!deviceSwitch) {
            Serial.println("❌ 设备开关关闭, 无法拍照");
        } else if (isCapturing) {
            Serial.println("❌ 设备正在拍照中, 请稍候");
        }
    }
}
}

```

```

// ===== 日程命令处理 =====
void handleScheduleCommand(JsonObject paras) {
    Serial.println("📅 处理日程更新命令");

    if (paras.containsKey("ScheduleTitle")) {
        scheduleTitle = paras["ScheduleTitle"].as<String>();
        Serial.printf("✅ 设置日程标题: %s\n", scheduleTitle.c_str());
    }
    if (paras.containsKey("ScheduleTime")) {
        scheduleTime = paras["ScheduleTime"].as<String>();
        Serial.printf("✅ 设置日程时间: %s\n", scheduleTime.c_str());
    }
    if (paras.containsKey("CountdownString")) {
        countdownString = paras["CountdownString"].as<String>();
        Serial.printf("✅ 设置倒计时: %s\n", countdownString.c_str());
    }

    lastActivity = "日程更新: " + scheduleTitle;
    Serial.println("✅ 日程数据更新完成");
}

// ===== 立即发送命令响应 =====
void sendCommandResponseImmediately(char* topic) {
    char requestId[100] = {0};
    char* pstr = topic;
    char* p = requestId;
    int flag = 0;

    while (*pstr) {
        if (flag) *p++ = *pstr;
        if (*pstr == '=') flag = 1;
        pstr++;
    }
    *p = '\0';

    char responseTopic[200] = {0};
    strcat(responseTopic, MQTT_TOPIC_CMD_RESPONSE);
    strcat(responseTopic, requestId);

    const char* responseData = "{\"result_code\": 0, \"response_name\": \"COMMAND_RESPONSE\", \"paras\": {\"result\": \"success\"}}";

    if (mqttClient.publish(responseTopic, responseData)) {
        Serial.println("✅ 命令响应已立即发送");
    } else {
        Serial.println("❌ 命令响应发送失败");
        errorCount++;
    }
}

// ===== AI结果命令处理 =====
void handleAIResultCommand(JsonObject paras) {
    Serial.println("🤖 处理AI识别结果命令");

    if (paras.containsKey("AIResult")) {
        aiResult = paras["AIResult"].as<String>();
        Serial.printf("✅ 设置AI结果: %s\n", aiResult.c_str());
    }

    hasAIResult = true;
}

```



```

aiDetectionCount++;
lastActivity = "AI识别: " + aiResult.substring(0, 20) + "...";

// ☑ 收到AI结果命令后立即单独上报AIResult
Serial.println("🌀 收到AI结果命令, 立即单独上报AIResult...");
reportAIResultOnly();

Serial.println("☑ AI结果存储完成");
}

// ===== 发送命令响应 =====
void sendCommandResponse(char* topic) {
    char requestId[100] = {0};
    char* pstr = topic;
    char* p = requestId;
    int flag = 0;

    while (*pstr) {
        if (flag) *p++ = *pstr;
        if (*pstr == '=') flag = 1;
        pstr++;
    }
    *p = '\0';

    char responseTopic[200] = {0};
    strcat(responseTopic, MQTT_TOPIC_CMD_RESPONSE);
    strcat(responseTopic, requestId);

    const char* responseData = "{\"result_code\": 0, \"response_name\": \"COMMAND_RESPONSE\", \"paras\": {\"result\": \"success\"}}";

    if (mqttClient.publish(responseTopic, responseData)) {
        Serial.println("☑ 命令响应发送成功");
    } else {
        Serial.println("✗ 命令响应发送失败");
        errorCount++;
    }
}

// ===== MQTT连接检查 =====
void mqtt_check_connect() {
    while (!mqttClient.connected()) {
        Serial.println("🌀 连接华为云MQTT...");

        if (mqttClient.connect(CLIENT_ID, MQTT_USER, MQTT_PASSWORD)) {
            Serial.println("☑ MQTT连接成功");
            mqttClient.subscribe(MQTT_TOPIC_COMMANDS);
            mqttClient.subscribe(MQTT_TOPIC_MESSAGES);
            Serial.println("☑ 主题订阅成功");
        } else {
            Serial.printf("✗ MQTT连接失败, 错误代码: %d, 3秒后重试...\n",
mqttClient.state());
            errorCount++;
            delay(3000);
        }
    }
}

// ===== 核心函数: 拍照并上传到云平台进行AI识别 =====
void captureAndSendForAI() {
    isCapturing = true;

```

```

Serial.println("\n🌀 开始AI识别流程...");

camera_fb_t *fb = esp_camera_fb_get();
if (!fb) {
    Serial.println("❌ 摄像头捕获失败");
    errorCount++;
    isCapturing = false;
    return;
}

Serial.printf("📷 获取到图片, 大小: %d bytes\n", fb->len);
photoCount++;

String imageBase64 = base64::encode(fb->buf, fb->len);
int totalLength = imageBase64.length();
int totalChunks = (totalLength + BASE64_CHUNK_SIZE - 1) / BASE64_CHUNK_SIZE;

Serial.printf("📦 Base64编码完成, 长度: %d, 分片数: %d\n", totalLength, totalChunks);

// 1. 发送开始标记
bool startSuccess = reportImageStart(totalChunks, fb->len);
mqttClient.loop();
delay(200);

if (!startSuccess) {
    Serial.println("❌ 开始标记发送失败, 中止流程");
    esp_camera_fb_return(fb);
    isCapturing = false;
    return;
}

// 2. 分片发送图片数据
for (int i = 0; i < totalChunks; i++) {
    if (!mqttClient.connected()) {
        Serial.println("❌ MQTT连接断开, 中止发送");
        break;
    }

    int startPos = i * BASE64_CHUNK_SIZE;
    int chunkLength = min(BASE64_CHUNK_SIZE, totalLength - startPos);
    String chunk = imageBase64.substring(startPos, startPos + chunkLength);

    reportImageChunk(chunk.c_str(), i + 1, totalChunks);

    for (int j = 0; j < 10; j++) {
        delay(10);
        mqttClient.loop();
    }
}

// 3. 发送完成标记
bool completeSuccess = reportImageComplete(totalChunks);
mqttClient.loop();

esp_camera_fb_return(fb);
isCapturing = false;

if (completeSuccess) {
    Serial.println("✅ 图片数据上传完成");
    lastActivity = "拍照完成";
} else {

```

```

        Serial.println("❌ 完成标记发送失败");
        lastActivity = "拍照失败";
    }
}

// ===== 上报开始标记 =====
bool reportImageStart(int totalChunks, int imageSize) {
    char properties[512];
    char jsonBuf[1024];

    sprintf(properties,
        "\"ImageAction\":\"start\", \"TotalChunks\":%d, \"ImageSize\":%d, \"Timestamp\":%lu, \"Format\":\"jpg\"",
        totalChunks, imageSize, millis());
    sprintf(jsonBuf, MQTT_BODY_FORMAT, properties);

    bool success = mqttClient.publish(MQTT_TOPIC_REPORT, jsonBuf);
    if (success) {
        Serial.println("✅ 开始标记发送成功");
    } else {
        Serial.println("❌ 开始标记发送失败");
        errorCount++;
    }
    return success;
}

// ===== 上报分片数据 =====
void reportImageChunk(const char* chunkData, int chunkIndex, int totalChunks) {
    char properties[2048];
    char jsonBuf[3072];

    sprintf(properties,
        "\"ImageAction\":\"chunk\", \"ChunkIndex\":%d, \"TotalChunks\":%d, \"ImageData\":\"%s\"",
        chunkIndex, totalChunks, chunkData);
    sprintf(jsonBuf, MQTT_BODY_FORMAT, properties);

    bool success = mqttClient.publish(MQTT_TOPIC_REPORT, jsonBuf);
    if (!success) {
        Serial.printf("❌ 分片 %d 发送失败\n", chunkIndex);
        errorCount++;
    }
}

// ===== 上报完成标记 =====
bool reportImageComplete(int totalChunks) {
    char properties[256];
    char jsonBuf[512];

    sprintf(properties,
        "\"ImageAction\":\"complete\", \"TotalChunks\":%d, \"Timestamp\":%lu",
        totalChunks, millis());
    sprintf(jsonBuf, MQTT_BODY_FORMAT, properties);

    bool success = mqttClient.publish(MQTT_TOPIC_REPORT, jsonBuf);
    if (success) {
        Serial.println("✅ 完成标记发送成功");
    } else {
        Serial.println("❌ 完成标记发送失败");
        errorCount++;
    }
    return success;
}

```

```

}

// ===== 系统状态上报（定时上报，不包含AIResult） =====
bool reportSystemStatus() {
    if (!mqttClient.connected()) {
        Serial.println("❌ MQTT未连接，无法上报状态");
        return false;
    }

    if (ESP.getFreeHeap() < 10000) {
        Serial.printf("⚠️ 内存不足: %d, 跳过状态上报\n", ESP.getFreeHeap());
        return false;
    }

    systemUptime = millis() / 1000;

    char properties[1500];
    char jsonBuf[2000];

    // ☑️ 定时上报: 图片中显示的所有变量，但不包含AIResult
    sprintf(properties,
        "\"Humidity\": \"%s\", \"",
        "\"FeelsLike\": \"%s\", \"",
        "\"ScheduleTitle\": \"%s\", \"",
        "\"WindScale\": \"%s\", \"",
        "\"Visibility\": \"%s\", \"",
        "\"CountdownString\": \"%s\", \"",
        "\"Temp\": \"%s\", \"",
        "\"WindDir\": \"%s\", \"",
        "\"Condition\": \"%s\", \"",
        "\"WindSpeed\": \"%s\", \"",
        "\"ScheduleTime\": \"%s\", \"",
        "\"SystemUptime\": %lu, \"",
        "\"PhotoCount\": %d, \"",
        "\"Timestamp\": %lu",
        humidity.c_str(),
        feelsLike.c_str(),
        scheduleTitle.c_str(),
        windScale.c_str(),
        visibility.c_str(),
        countdownString.c_str(),
        temp.c_str(),
        windDir.c_str(),
        condition.c_str(),
        windSpeed.c_str(),
        scheduleTime.c_str(),
        systemUptime,
        photoCount,
        millis());

    sprintf(jsonBuf, MQTT_BODY_FORMAT, properties);

    Serial.printf("📶 定时上报系统状态，长度: %d\n", strlen(jsonBuf));

    bool success = mqttClient.publish(MQTT_TOPIC_REPORT, jsonBuf);
    if (success) {
        Serial.println("☑️ 系统状态上报成功");
    } else {
        Serial.println("❌ 系统状态上报失败");
        errorCount++;
    }
}

```

```

    return success;
}

// ===== AI识别结果单独上报（仅AI结果命令后调用） =====
bool reportAIResultOnly() {
    if (!mqttClient.connected()) {
        Serial.println("❌ MQTT未连接，无法上报AI结果");
        return false;
    }

    if (ESP.getFreeHeap() < 8000) {
        Serial.printf("⚠️ 内存不足: %d, 跳过AI结果上报\n", ESP.getFreeHeap());
        return false;
    }

    if (!hasAIResult || aiResult.isEmpty()) {
        Serial.println("⚠️ 无AI识别结果可上报");
        return false;
    }

    // 准备AI结果（安全截断）
    String aiResultForReport = aiResult;
    if (aiResultForReport.length() > 1000) {
        aiResultForReport = aiResultForReport.substring(0, 1000) + "...";
    }

    // 清理JSON字符串中的特殊字符
    String safeAIResult = escapeJSONString(aiResultForReport);

    char properties[1500];
    char jsonBuf[2000];

    // ☑️ 只上报AIResult
    sprintf(properties,
        "\"AIResult\": \"%s\",",
        "\"AIResultType\": \"%s\",",
        "\"AITimestamp\": %lu,",
        "\"MessageType\": \"AI_Result\",",
        "\"Timestamp\": %lu",
        safeAIResult.c_str(),
        aiResultType.c_str(),
        aiTimestamp,
        millis());

    sprintf(jsonBuf, MQTT_BODY_FORMAT, properties);

    Serial.printf("📡 单独上报AI识别结果，长度: %d\n", strlen(jsonBuf));
    Serial.println("📄 AI结果内容预览:");
    Serial.println(aiResultForReport.substring(0, 100) + "...");

    bool success = mqttClient.publish(MQTT_TOPIC_REPORT, jsonBuf);

    if (success) {
        Serial.println("☑️ AI识别结果单独上报成功");
        lastAIReportTime = millis();
        lastActivity = "AI结果上报";
    } else {
        Serial.println("❌ AI识别结果上报失败");
        errorCount++;
    }
}

```

```

    return success;
}

// ===== JSON字符串转义函数 =====
String escapeJSONString(const String& input) {
    String output;
    output.reserve(input.length() + 10);

    for (size_t i = 0; i < input.length(); i++) {
        char c = input.charAt(i);
        switch (c) {
            case '"': output += "\\\""; break;
            case '\\': output += "\\\\"; break;
            case '\b': output += "\\b"; break;
            case '\f': output += "\\f"; break;
            case '\n': output += "\\n"; break;
            case '\r': output += "\\r"; break;
            case '\t': output += "\\t"; break;
            default: output += c; break;
        }
    }

    return output;
}

```

华为云Functionflow代码:

(1) image_analysis函数

```

import json
import requests
import base64
import time
from collections import defaultdict

# 配置信息
CONFIG = {
    "maas": {
        "api_url": "https://api.modelarts-maas.com/v1/chat/completions",
        "model_name": "qwen2.5-v1-72b",
        "api_key": "3CwWbwZM01viqZgaeIq5dKO_mesibqar1AhNgUQZaAMXceXbJsfgmmZnUiz0d5w-1hchktuR4Y0r0Vn_DCucRcQ"
    },
    "iot": {
        "username": "CAM01",
        "password": "lego0369",
        "domain_name": "bricksburgakataka",
        "project_id": "2c1b69ce3828429fb0682557f8309123",
        "region": "cn-east-3",
        "iam_https": "iam.cn-east-3.myhuaweicloud.com",
        "iotda_https": "943f583bff.st1.iotda-app.cn-east-3.myhuaweicloud.com"
    }
}

# 存储图片数据
image_data_store = defaultdict(str)

def process_esp32_image(event, context):
    """

```

处理ESP32图片并通过命令下发AI结果

"""

print("🚀 开始处理ESP32图片数据")

try:

解析输入数据

body = event

if 'notify_data' in event:

body = event['notify_data']['body']

print("🐞 检测到IoTDA转发格式")

提取设备信息

device_id = body.get('device_id', 'unknown_device')

if device_id == 'unknown_device' and 'notify_data' in event:

device_id = event['notify_data']['header'].get('device_id',
'unknown_device')

services = body.get('services', [])

if not services:

return build_response(400, "无效数据格式")

properties = services[0].get('properties', {})

image_action = properties.get('ImageAction')

print(f"🐞 设备: {device_id}, 动作: {image_action}")

处理分片逻辑

if image_action == 'start':

image_data_store[device_id] = ""

return build_response(200, "开始接收图片")

elif image_action == 'chunk':

chunk_data = properties.get('ImageData', '')

if chunk_data:

image_data_store[device_id] += chunk_data

chunk_index = properties.get('ChunkIndex', 0)

print(f"📦 接收分片 {chunk_index}, 当前长度:

{len(image_data_store[device_id])}")

return build_response(200, "分片接收成功")

else:

return build_response(400, "分片数据为空")

elif image_action == 'complete':

full_image_data = image_data_store[device_id]

if not full_image_data:

return build_response(400, "无图片数据")

print(f"🔗 开始AI识别, 数据长度: {len(full_image_data)}")

调用MaaS API进行AI识别

ai_result = call_maas_api(full_image_data)

通过命令下发AI结果到ESP32

send_success = send_ai_result_via_command(device_id, ai_result)

清理数据

if device_id in image_data_store:

del image_data_store[device_id]

if send_success:

return build_response(200, "AI识别完成, 结果已通过命令下发到ESP32")

```

        else:
            return build_response(200, "AI识别完成，但发送到ESP32失败")

    else:
        return build_response(400, f"未知动作: {image_action}")

except Exception as e:
    print(f"❌ 处理错误: {str(e)}")
    return build_response(500, f"处理失败: {str(e)}")

def call_maas_api(image_base64):
    """调用MaaS API进行图像识别"""
    try:
        url = CONFIG["maas"]["api_url"]
        api_key = CONFIG["maas"]["api_key"]

        headers = {
            'Content-Type': 'application/json',
            'Authorization': f'Bearer {api_key}'
        }

        data = {
            "model": CONFIG["maas"]["model_name"],
            "messages": [
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "text",
                            "text": "观察照片中人物在干什么，若探测不到人物，则观察环境。用
中文回答。"
                        },
                        {
                            "type": "image_url",
                            "image_url": {
                                "url": f"data:image/jpeg;base64,{image_base64}"
                            }
                        }
                    ]
                }
            ],
            "max_tokens": 800,
            "temperature": 0.7
        }

        print(f"📤 发送AI识别请求...")

        response = requests.post(url, headers=headers, json=data, timeout=60)

        if response.status_code == 200:
            result = response.json()
            if 'choices' in result and len(result['choices']) > 0:
                content = result['choices'][0]['message']['content']
                print(f"✅ AI识别成功，长度: {len(content)} 字符")
                return content
            else:
                return "AI分析完成，但返回格式异常"
        else:
            print(f"❌ AI服务调用失败: {response.status_code}")
            return f"AI服务调用失败: {response.status_code}"

```



```

except Exception as e:
    print(f"❌ AI服务异常: {str(e)}")
    return f"AI服务异常: {str(e)}"

def send_ai_result_via_command(device_id, ai_result):
    """
    通过命令下发方式发送AI结果到ESP32
    """
    try:
        # 获取IAM Token
        token = get_iam_token()
        if not token:
            print(f"❌ 无法获取IAM Token")
            return False

        # IoTDA命令下发URL
        iot_url = f"https://{CONFIG['iot']['iotda_https']}/v5/iot/{CONFIG['iot']
['project_id']}/devices/{device_id}/commands"

        headers = {
            "Content-Type": "application/json",
            "X-Auth-Token": token
        }

        # 智能截断, 避免消息过长
        if len(ai_result) > 1000:
            truncated = ai_result[:1000]
            last_period = truncated.rfind('.')
            if last_period > 500:
                truncated = truncated[:last_period + 1]
            truncated += f"\n(内容过长已截断, 原始长度:{len(ai_result)}字符)"
        else:
            truncated = ai_result

        # 构建命令数据 - 使用分立变量格式
        command_data = {
            "command_name": "AIResult",
            "paras": {
                "AIResult": truncated,
                "ResultType": "image_analysis",
                "Timestamp": int(time.time())
            },
            "service_id": "ESP32CAM01"
        }

        print(f"📬 通过命令下发AI结果到ESP32: {device_id}")
        print(f"目标URL: {iot_url}")
        print(f"命令内容: {json.dumps(command_data, ensure_ascii=False, indent=2)}")

        response = requests.post(iot_url, json=command_data, headers=headers,
                                timeout=30)

        if response.status_code in [200, 201, 204]:
            print(f"✅ AI结果通过命令下发成功")
            return True
        else:
            print(f"❌ 命令下发失败: {response.status_code} - {response.text}")
            return False

    except Exception as e:
        print(f"❌ 命令下发异常: {str(e)}")

```

```

        return False

def get_iam_token():
    """获取IAM Token"""
    try:
        iam_url = f"https://{CONFIG['iot']['iam_https']}/v3/auth/tokens"

        iam_body = {
            "auth": {
                "identity": {
                    "methods": ["password"],
                    "password": {
                        "user": {
                            "name": CONFIG["iot"]["username"],
                            "password": CONFIG["iot"]["password"],
                            "domain": {
                                "name": CONFIG["iot"]["domain_name"]
                            }
                        }
                    }
                },
                "scope": {
                    "project": {
                        "name": "cn-east-3"
                    }
                }
            }
        }

        headers = {
            "Content-Type": "application/json"
        }

        print("🔑 获取IAM Token...")

        response = requests.post(iam_url, json=iam_body, headers=headers, timeout=30)

        if response.status_code == 201:
            token = response.headers.get('X-Subject-Token')
            if token:
                print("✅ IAM Token获取成功")
                return token
            else:
                print("❌ 响应头中未找到X-Subject-Token")
                return None
        else:
            print(f"❌ IAM Token获取失败: {response.status_code}")
            return None

    except Exception as e:
        print(f"❌ IAM Token获取异常: {str(e)}")
        return None

def build_response(status_code, message):
    """构建响应"""
    return {
        "statusCode": status_code,
        "body": json.dumps({"message": message}, ensure_ascii=False),
        "headers": {
            "Content-Type": "application/json"
        }
    }

```

```
}
```

(2) ScheduleRemind函数

```
import json
import requests
import time
from datetime import datetime, timedelta
from obs import ObsClient

# 配置信息
CONFIG = {
    "iot": {
        "username": "CAM01",
        "password": "lego0369",
        "domain_name": "bricksburgakataka",
        "project_id": "2c1b69ce3828429fb0682557f8309123",
        "region": "cn-east-3",
        "iam_https": "iam.cn-east-3.myhuaweicloud.com",
        "iotda_https": "943f583bff.st1.iotda-app.cn-east-3.myhuaweicloud.com"
    },
    "obs": {
        "ak": "HPUA1JTKX1WIAIAIFH01",
        "sk": "z81ZDTWOTJERZh3JwwfyGbe4ihs5GR7W0h02rt01",
        "endpoint": "obs.cn-east-3.myhuaweicloud.com",
        "bucket": "esp32cam-aktk",
        "ical_file": "calendar.ics"
    },
    "schedule": {
        "check_interval": 5, # 5秒检查一次（测试用）
        "timezone": "Asia/Shanghai"
    }
}

# 全局变量
last_check_time = 0

def handler(event, context):
    """
    主处理函数 - 华为云函数标准入口
    """
    print("=" * 50)
    print("📅 开始日程检查...")
    print("=" * 50)

    global last_check_time

    try:
        # 检查是否达到检查间隔
        current_time = time.time()
        if current_time - last_check_time < CONFIG["schedule"]["check_interval"]:
            print("🕒 未到检查时间，跳过")
            return build_response(200, "未到检查时间")

        last_check_time = current_time

        # 获取设备ID
        device_id = extract_device_id(event)
        if not device_id:
            device_id = "690343cb0a9ab42ae589dd21_CAM01" # 默认设备ID
```

```

print(f"🔗 目标设备: {device_id}")

# 从OBS获取并解析iCal文件
upcoming_events = get_ical_from_obs()

if upcoming_events:
    # 找到离当前时间最近的任务（包括所有未来任务）
    nearest_event = find_nearest_event(upcoming_events)

    if nearest_event:
        # 发送命令到设备
        if send_schedule_command(device_id, nearest_event):
            message = f"成功发送最近任务提醒: {nearest_event['title']} (剩余{nearest_event['countdown_string']})"
            print(f"✅ {message}")
            return build_response(200, message)
        else:
            return build_response(500, "发送任务命令失败")
    else:
        print("❗ 没有找到未来的任务")
        # 发送空任务命令，清空设备上的任务
        send_empty_schedule_command(device_id)
        return build_response(200, "没有未来的任务")
else:
    print("❗ 没有找到任何日程")
    # 发送空任务命令，清空设备上的任务
    send_empty_schedule_command(device_id)
    return build_response(200, "没有日程")

except Exception as e:
    print(f"❌ 日程处理错误: {str(e)}")
    import traceback
    print(f"❌ 详细堆栈: {traceback.format_exc()}")
    return build_response(500, f"日程处理失败: {str(e)}")

def find_nearest_event(events):
    """
    找到离当前时间最近的任务（包括所有未来任务）
    """
    if not events:
        return None

    current_time = datetime.now()
    nearest_event = None
    min_time_diff = float('inf')

    print(f"🕒 开始查找最近任务，当前时间: {current_time.strftime('%Y-%m-%d %H:%M:%S')}")

    for event in events:
        try:
            # 解析事件时间（格式: "01-13 10:30"）
            event_time_str = event['start_time']
            now = datetime.now()

            # 解析月-日 时:分
            month_day, time_part = event_time_str.split(' ')
            month, day = map(int, month_day.split('-'))
            hour, minute = map(int, time_part.split(':'))

```

```

# 创建日期时间对象（使用当前年份）
event_time = datetime(now.year, month, day, hour, minute)

# 如果事件时间已经过去（比如去年的事件），则使用下一年的日期
if event_time < current_time:
    event_time = datetime(now.year + 1, month, day, hour, minute)

# 计算时间差（秒）
time_diff = (event_time - current_time).total_seconds()

print(f"📅 事件 '{event['title']}': {event_time.strftime('%Y-%m-%d %H:%M:%S')}, 时间差: {time_diff/60:.1f}分钟")

# 只考虑未来的事件，找到时间差最小的
if time_diff >= 0 and time_diff < min_time_diff:
    min_time_diff = time_diff
    nearest_event = event.copy() # 创建副本
    nearest_event['countdown_minutes'] = int(time_diff / 60) # 转换为分钟
    nearest_event['countdown_string'] =
format_countdown_string(int(time_diff / 60)) # 转换为字符串格式
    nearest_event['exact_time'] = event_time.strftime("%Y-%m-%d %H:%M:%S")

except Exception as e:
    print(f"❌ 处理事件时间错误: {event}, 错误: {e}")
    continue

if nearest_event:
    print(f"🕒 找到最近任务: '{nearest_event['title']}', 剩余
{nearest_event['countdown_string']}")
else:
    print("❌ 没有找到未来的任务")

return nearest_event

def format_countdown_string(total_minutes):
    """
    将分钟数转换为 *h*m 的字符串格式
    例如: 125分钟 -> "2h5m", 30分钟 -> "30m", 90分钟 -> "1h30m"
    """
    if total_minutes < 60:
        return f"{total_minutes}m"
    else:
        hours = total_minutes // 60
        minutes = total_minutes % 60
        if minutes == 0:
            return f"{hours}h"
        else:
            return f"{hours}h{minutes}m"

def send_schedule_command(device_id, event_info):
    """
    发送日程命令到ESP32设备
    """
    try:
        print(f"\n📡 准备发送任务命令到设备 {device_id}")
        print(f"📋 最近任务详情: {event_info}")

        token = get_iam_token()
        if not token:
            print("❌ 无法获取IAM Token")
            return False
    
```

```

# IoTDA命令下发URL
iot_url = f"https://{CONFIG['iot']['iotda_https']}/v5/iot/{CONFIG['iot']
['project_id']}/devices/{device_id}/commands"

headers = {
    "Content-Type": "application/json",
    "X-Auth-Token": token
}

# 构建命令数据 - 使用字符串格式的倒计时
command_data = {
    "service_id": "ESP32CAM01",
    "command_name": "ScheduleUpdate",
    "paras": {
        "HasSchedule": True,
        "ScheduleTitle": event_info['title'],
        "ScheduleTime": event_info['start_time'],
        "CountdownString": event_info['countdown_string'], # 改为字符串格式
        "ExactTime": event_info.get('exact_time', ''),
        "Location": event_info.get('location', ''),
        "Description": event_info.get('description', ''),
        "UpdateTime": datetime.now().strftime("%m-%d %H:%M")
    }
}

print(f"✉ 发送任务命令到设备: {device_id}")
print(f"📌 目标URL: {iot_url}")
print(f"💬 命令内容: {json.dumps(command_data, indent=2,
ensure_ascii=False)}")

response = requests.post(iot_url, json=command_data, headers=headers,
timeout=30)

if response.status_code in [200, 201, 204]:
    print("✅ 任务命令发送成功")
    return True
else:
    print(f"❌ 任务命令发送失败: {response.status_code}")
    print(f"❌ 错误详情: {response.text}")
    return False

except Exception as e:
    print(f"❌ 发送任务命令异常: {str(e)}")
    import traceback
    print(f"❌ 详细堆栈: {traceback.format_exc()}")
    return False

def send_empty_schedule_command(device_id):
    """
    发送空任务命令，清空设备上的任务
    """
    try:
        print(f"🗑 发送空任务命令，清空设备任务")

        token = get_iam_token()
        if not token:
            return False

        iot_url = f"https://{CONFIG['iot']['iotda_https']}/v5/iot/{CONFIG['iot']
['project_id']}/devices/{device_id}/commands"

```

```

headers = {
    "Content-Type": "application/json",
    "X-Auth-Token": token
}

command_data = {
    "service_id": "ESP32CAM01",
    "command_name": "ScheduleUpdate",
    "paras": {
        "HasSchedule": False,
        "ScheduleTitle": "",
        "ScheduleTime": "",
        "CountdownString": "", # 改为字符串格式
        "ExactTime": "",
        "Location": "",
        "Description": "",
        "UpdateTime": datetime.now().strftime("%m-%d %H:%M")
    }
}

response = requests.post(iot_url, json=command_data, headers=headers,
timeout=30)

if response.status_code in [200, 201, 204]:
    print("✅ 空任务命令发送成功")
    return True
else:
    print(f"❌ 空任务命令发送失败: {response.status_code}")
    return False

except Exception as e:
    print(f"❌ 发送空任务命令异常: {str(e)}")
    return False

# 其他函数保持不变 (get_ical_from_obs, parse_all_future_events,
process_event_for_future,
# extract_summary, parse_ical_datetime_fixed, extract_device_id, get_iam_token,
build_response)

def get_ical_from_obs():
    """
    从OBS获取iCal文件并解析 - 获取所有未来事件
    """
    try:
        print("📁 从OBS获取iCal文件...")

        # 初始化OBS客户端
        obs_client = ObsClient(
            access_key_id=CONFIG["obs"]["ak"],
            secret_access_key=CONFIG["obs"]["sk"],
            server=CONFIG["obs"]["endpoint"]
        )

        # 获取iCal文件
        resp = obs_client.getObject(
            bucketName=CONFIG["obs"]["bucket"],
            objectKey=CONFIG["obs"]["ical_file"]
        )

        if resp.status < 300:

```

```

        ical_content = resp.body.response.read().decode('utf-8')
        print(f"✅ 成功从OBS获取iCal文件")
        print(f"📄 文件大小: {len(ical_content)} bytes")

        # 解析所有未来事件
        return parse_all_future_events(ical_content)
    else:
        print(f"❌ 从OBS获取文件失败: 状态码: {resp.status}")
        return []

except Exception as e:
    print(f"❌ OBS操作异常: {str(e)}")
    return []

def parse_all_future_events(ical_content):
    """
    解析所有未来事件（不限制时间范围）
    """
    future_events = []

    try:
        current_time = datetime.now()

        print(f"🕒 当前时间: {current_time.strftime('%Y-%m-%d %H:%M:%S')}")
        print(f"🔍 查找所有未来事件...")

        # 解析逻辑
        lines = ical_content.split('\n')
        event_data = {}
        in_event = False
        event_count = 0

        for line in lines:
            line = line.strip()

            if line == 'BEGIN:VEVENT':
                event_data = {}
                in_event = True
                event_count += 1
            elif line == 'END:VEVENT':
                in_event = False
                # 处理完整的事件
                if event_data:
                    event_info = process_event_for_future(event_data, current_time)
                    if event_info:
                        future_events.append(event_info)
                event_data = {}
            elif in_event and ':' in line:
                # 处理带参数的字段
                parts = line.split(':', 1)
                if len(parts) == 2:
                    key_part, value = parts
                    key = key_part.split(';')[0] # 提取主字段名
                    event_data[key] = value

        print(f"📊 总共解析了 {event_count} 个事件, 找到 {len(future_events)} 个未来事件")

    except Exception as e:
        print(f"❌ 解析iCal内容异常: {str(e)}")

```



```

    return future_events

def process_event_for_future(event_data, current_time):
    """
    处理事件数据 - 只筛选未来事件
    """
    try:
        summary = extract_summary(event_data)

        # 解析开始时间
        dtstart = event_data.get('DTSTART', '')
        if not dtstart:
            return None

        start_time = parse_ical_datetime_fixed(dtstart)
        if not start_time:
            return None

        # 如果事件时间已经过去（比如去年的事件），则使用下一年的日期
        if start_time < current_time:
            start_time = datetime(current_time.year + 1, start_time.month,
start_time.day,
                                start_time.hour, start_time.minute, start_time.second)

        # 只返回未来事件
        if start_time >= current_time:
            return {
                "title": summary,
                "start_time": start_time.strftime("%m-%d %H:%M"),
                "location": event_data.get('LOCATION', ''),
                "description": event_data.get('DESCRIPTION', '')[:50]
            }

    except Exception as e:
        print(f"❌ 处理事件数据异常: {str(e)}")

    return None

def extract_summary(event_data):
    """提取事件标题"""
    if 'SUMMARY' in event_data:
        return event_data['SUMMARY'].strip() or '无标题'
    return '无标题'

def parse_ical_datetime_fixed(dt_string):
    """
    修复后的时间解析 - 处理多种iCal时间格式
    """
    try:
        # 处理带时区的格式: 20260113T103000
        if 'T' in dt_string and len(dt_string) >= 15:
            # 移除可能的时区后缀
            dt_clean = dt_string
            if 'Z' in dt_clean:
                dt_clean = dt_clean.replace('Z', '')

            # 解析基础格式: YYYYMMDDTHHMMSS
            if len(dt_clean) == 15:
                year = int(dt_clean[0:4])
                month = int(dt_clean[4:6])
                day = int(dt_clean[6:8])

```

```

        hour = int(dt_clean[9:11])
        minute = int(dt_clean[11:13])
        second = int(dt_clean[13:15])

        return datetime(year, month, day, hour, minute, second)

# 处理全天事件格式: YYYYMMDD
elif len(dt_string) == 8 and dt_string.isdigit():
    year = int(dt_string[0:4])
    month = int(dt_string[4:6])
    day = int(dt_string[6:8])
    return datetime(year, month, day, 9, 0, 0) # 默认上午9点

except Exception as e:
    print(f"❌ 解析日期时间异常: {dt_string}, 错误: {e}")

return None

def extract_device_id(event):
    """从事件中提取设备ID"""
    try:
        if isinstance(event, dict) and 'notify_data' in event:
            return event['notify_data']['header'].get('device_id')
        elif isinstance(event, dict) and 'device_id' in event:
            return event['device_id']
        else:
            # 尝试解析JSON字符串
            if isinstance(event, str):
                event_data = json.loads(event)
                return extract_device_id(event_data)
    except Exception as e:
        print(f"⚠️ 提取设备ID失败: {e}")
    return None

def get_iam_token():
    """获取IAM Token"""
    try:
        print("🔑 获取IAM Token...")

        iam_url = f"https://{CONFIG['iot']['iam_https']}/v3/auth/tokens"

        iam_body = {
            "auth": {
                "identity": {
                    "methods": ["password"],
                    "password": {
                        "user": {
                            "name": CONFIG["iot"]["username"],
                            "password": CONFIG["iot"]["password"],
                            "domain": {
                                "name": CONFIG["iot"]["domain_name"]
                            }
                        }
                    }
                }
            },
            "scope": {
                "project": {
                    "name": "cn-east-3"
                }
            }
        }
    
```

```

    }

    headers = {
        "Content-Type": "application/json"
    }

    response = requests.post(iam_url, json=iam_body, headers=headers, timeout=30)

    if response.status_code == 201:
        token = response.headers.get('X-Subject-Token')
        if token:
            print("☑ IAM Token获取成功")
            return token
        else:
            print("✗ 响应头中未找到X-Subject-Token")
            return None
    else:
        print(f"✗ IAM Token获取失败: {response.status_code}")
        return None

except Exception as e:
    print(f"✗ IAM Token获取异常: {str(e)}")
    return None

def build_response(status_code, message):
    """构建响应"""
    return {
        "statusCode": status_code,
        "body": json.dumps({"message": message}, ensure_ascii=False),
        "headers": {
            "Content-Type": "application/json"
        }
    }
}

```

(3) Weather函数

```

import json
import requests
import time
from datetime import datetime

# 和风天气配置 - 商业版
WEATHER_CONFIG = {
    "api_key": "0f9b2eb09b204919a5a0d862908f48da",
    "api_host": "m23jpg673v.re.qweatherapi.com",
    "location": "101210101", # 杭州
    "device_id": "690343cb0a9ab42ae589dd21_CAM01",

    "iot": {
        "username": "CAM01",
        "password": "lego0369",
        "domain_name": "bricksburgakataka",
        "project_id": "2c1b69ce3828429fb0682557f8309123",
        "region": "cn-east-3",
        "iam_https": "iam.cn-east-3.myhuaweicloud.com",
        "iotda_https": "943f583bff.st1.iotda-app.cn-east-3.myhuaweicloud.com"
    },
    "check_interval": 300 # 5分钟
}

```

```

def handler(event, context):
    """
    和风天气API处理函数 - 下发分立变量版本
    """
    print("🌩️ 和风天气服务启动 - 分立变量版本")

    try:
        # 获取设备ID
        device_id = extract_device_id_from_event(event) or WEATHER_CONFIG["device_id"]
        print(f"🔗 目标设备: {device_id}")

        # 获取天气数据
        weather_data = get_hefeng_commercial_weather()
        if not weather_data:
            return build_response(500, "获取天气数据失败")

        # 发送到设备（分立变量格式）
        if send_weather_to_device(device_id, weather_data):
            city_name = get_location_name(WEATHER_CONFIG['location'])
            return build_response(200, f"天气发送成功: {city_name}")
        else:
            return build_response(500, "天气发送失败")

    except Exception as e:
        print(f"❌ 天气处理错误: {str(e)}")
        import traceback
        traceback.print_exc()
        return build_response(500, f"处理失败: {str(e)}")

def send_weather_to_device(device_id, weather_data):
    """
    发送天气信息到ESP32设备 - 分立变量格式
    """
    try:
        print(f"🚀 准备发送天气信息到设备 {device_id}")

        token = get_iam_token()
        if not token:
            print("❌ 无法获取IAM Token")
            return False

        iot_url = f"https://{WEATHER_CONFIG['iot']}["
        iotda_https = WEATHER_CONFIG['iotda_https']
        v5_iot = WEATHER_CONFIG['v5_iot']
        project_id = WEATHER_CONFIG['project_id']
        device_id = device_id
        commands = commands

        headers = {
            "Content-Type": "application/json",
            "X-Auth-Token": token
        }

        # 构建分立变量命令
        command_data = build_weather_command(weather_data)

        print(f"✉️ 发送命令到IoT平台...")
        print(f"📄 命令内容: {json.dumps(command_data, ensure_ascii=False, indent=2)}")

        response = requests.post(iot_url, json=command_data, headers=headers,
                                timeout=30)

        if response.status_code in [200, 201, 204]:

```

```

        print("☑ 天气命令发送成功")
        return True
    else:
        print(f"✗ 发送失败: {response.status_code}")
        print(f"✗ 错误响应: {response.text}")
        return False

except Exception as e:
    print(f"✗ 发送天气信息异常: {str(e)}")
    return False

def build_weather_command(weather_data):
    """
    构建分立变量的天气命令
    """
    now = weather_data["now"]

    # 获取城市名称
    city_name = get_location_name(WEATHER_CONFIG['location'])

    # 构建分立变量参数
    paras = {
        "City": city_name,
        "Temp": now.get('temp', 'N/A'),
        "Condition": now.get('text', 'N/A'),
        "Humidity": now.get('humidity', 'N/A'),
        "WindScale": now.get('windScale', 'N/A'),
        "WindDir": now.get('windDir', 'N/A'),
        "WindSpeed": now.get('windSpeed', 'N/A'),
        "FeelsLike": now.get('feelsLike', 'N/A'),
        "Visibility": now.get('vis', 'N/A'),
        "UpdateTime": datetime.now().strftime("%H:%M")
    }

    command_data = {
        "command_name": "WeatherUpdate",
        "paras": paras,
        "service_id": "ESP32CAM01"
    }

    return command_data

# 其他辅助函数保持不变
def get_hefeng_commercial_weather():
    """获取商业版和风天气数据"""
    try:
        print("📡 请求商业版和风天气API...")

        url = f"https://{WEATHER_CONFIG['api_host']}/v7/weather/now"
        params = {
            "key": WEATHER_CONFIG["api_key"],
            "location": WEATHER_CONFIG["location"],
            "lang": "zh",
            "unit": "m"
        }

        headers = {
            "User-Agent": "HuaweiCloudFunction/1.0",
            "Accept": "application/json"
        }

```

```

response = requests.get(url, params=params, headers=headers, timeout=15)

if response.status_code == 200:
    data = response.json()
    code = data.get("code")
    if code == "200":
        print("🌤️ 商业版天气数据获取成功!")
        return data
    else:
        error_msg = data.get("message", "未知错误")
        print(f"❌ API业务错误: code={code}, message={error_msg}")
        return None
else:
    print(f"❌ HTTP错误: {response.status_code}")
    return None

except Exception as e:
    print(f"❌ 请求异常: {str(e)}")
    return None

def get_iam_token():
    """获取IAM Token"""
    try:
        print("📄 获取IAM Token...")

        iam_url = f"https://{WEATHER_CONFIG['iot']['iam_https']}/v3/auth/tokens"

        iam_body = {
            "auth": {
                "identity": {
                    "methods": ["password"],
                    "password": {
                        "user": {
                            "name": WEATHER_CONFIG["iot"]["username"],
                            "password": WEATHER_CONFIG["iot"]["password"],
                            "domain": {
                                "name": WEATHER_CONFIG["iot"]["domain_name"]
                            }
                        }
                    }
                }
            },
            "scope": {
                "project": {
                    "name": "cn-east-3"
                }
            }
        }

        headers = {
            "Content-Type": "application/json"
        }

        response = requests.post(iam_url, json=iam_body, headers=headers, timeout=30)

        if response.status_code == 201:
            token = response.headers.get('X-Subject-Token')
            if token:
                print("✅ IAM Token获取成功")
                return token
            else:

```

```

        print("❌ 响应头中未找到X-Subject-Token")
        return None
    else:
        print(f"❌ IAM Token获取失败: {response.status_code}")
        return None

except Exception as e:
    print(f"❌ IAM Token获取异常: {str(e)}")
    return None

def get_location_name(location_id):
    """根据位置ID获取位置名称"""
    locations = {
        "101210101": "杭州",
        "101010100": "北京",
        "101020100": "上海",
        "101280101": "广州",
        "101280601": "深圳"
    }
    return locations.get(location_id, f"城市{location_id}")

def extract_device_id_from_event(event):
    """从事件中提取设备ID"""
    try:
        if isinstance(event, dict):
            if 'notify_data' in event:
                return event['notify_data']['header'].get('device_id')
            elif 'device_id' in event:
                return event['device_id']
        elif isinstance(event, str):
            event_data = json.loads(event)
            return extract_device_id_from_event(event_data)
    except Exception as e:
        print(f"⚠️ 提取设备ID失败: {e}")
    return None

def build_response(status_code, message):
    """构建响应"""
    return {
        "statusCode": status_code,
        "body": json.dumps({"message": message}, ensure_ascii=False),
        "headers": {
            "Content-Type": "application/json"
        }
    }

# 测试函数
def test_weather_command():
    """测试天气命令构建"""
    test_data = {
        "now": {
            "temp": "25",
            "text": "晴",
            "humidity": "65",
            "windScale": "3",
            "windDir": "东南风",
            "windSpeed": "15",
            "feelsLike": "26",
            "vis": "10"
        }
    }

```

```
command = build_weather_command(test_data)
print("测试命令结构:")
print(json.dumps(command, ensure_ascii=False, indent=2))

if __name__ == "__main__":
    test_weather_command()
```